Artificial Intelligence Search Techniques

Search is a fundamental technique in **Artificial Intelligence (AI)** for **problem-solving**, especially in areas like pathfinding, game playing, and planning.

Categories of Search Techniques in AI

Search techniques are broadly divided into:

1. **Uninformed (Blind) Search**
2. **Informed (Heuristic) Search**

1 Uninformed (Blind) Search

These algorithms **do not have any domain-specific knowledge**—they explore the search space blindly.

Technique	Description	Data Structure	Optimal	Complete
Breadth-First Search (BFS)	Explores level by level	Queue	□ (uniform cost)	□
Depth-First Search (DFS)	Explores depth-wise	Stack/Recursion	□	□
Uniform Cost Search (UCS)	Chooses node with lowest path cost	Priority Queue	□	□
Depth-Limited Search	DFS with a depth limit	Stack	□	□ (if depth limit ≥ goal)
Iterative Deepening DFS (IDDFS)	DFS with increasing depth limits	Stack	□	□

2 Informed (Heuristic) Search

These use **domain knowledge (heuristics)** to find solutions **faster**.

Technique	Description	Uses Heuristic?	Optimal	Complete
Best-First Search	Chooses most promising node based on `h(n)`	□	□	□
Greedy Search	Chooses node with lowest estimated cost to goal `h(n)`	□	□	□
A* Search	Chooses node with lowest `f(n) = g(n) + h(n)`	□	□ (if h is admissible)	□
Hill Climbing	Greedy local search that moves in direction of increase/decrease	□	□	□

Complete

An algorithm is **complete** if it is **guaranteed to find a solution** (if one exists).

- **Example:**
  - **BFS** is complete because it explores all levels and will eventually reach the goal if it exists.
  - **DFS** is **not complete** if there are infinite paths (e.g., loops).

Optimal

An algorithm is **optimal** if it **finds the best (least-cost or shortest) solution**.

- **Example:**
  - **BFS** is optimal **only if all steps have equal cost**.
  - **A*** is optimal **if the heuristic is admissible and consistent**.
  - **Greedy Search** is **not optimal**; it may find a suboptimal (shorter-looking) path.

Key Concepts:

➢ State Space

All possible configurations of a problem.

➢ Initial State

Starting point of the search.

➢ Goal Test

Checks if a state satisfies the goal condition.

➢ Successor Function

Generates next possible states from the current state.

➢ Cost Function

Total cost to reach a state (used in UCS, A*).

➢ Heuristic Function `h(n)`

Estimate of cost from node `n` to goal. Should be:

- **Admissible**: Never overestimates the true cost
- **Consistent**: `h(n) ≤ cost(n→n') + h(n')`

1. Breadth-First Search (BFS)

- **Strategy:** Explores all nodes at the current depth before moving to the next.
- **Data Structure:** Queue (FIFO)
- **Complete:** Yes (guarantees solution if one exists)
- **Optimal:** Yes, if all step costs are equal
- **Time/Space Complexity:** O(b^d)
  b = branching factor, d = depth of shallowest solution

Use Case: Shortest path problems (e.g., unweighted graph)

2. Depth-First Search (DFS)

- **Strategy:** Explores as far as possible along a branch before backtracking.
- **Data Structure:** Stack (LIFO) or recursion
- **Complete:** No (can get stuck in infinite paths)
- **Optimal:** No
- **Time/Space Complexity:** O(b^m)
  m = maximum depth

Use Case: Puzzle solving with large depth but limited memory

3. A* Search

- **Strategy:** Uses both path cost and heuristic to guide the search
- **Evaluation Function:** f(n) = g(n) + h(n)
  - *g(n)*: cost from start to n
  - *h(n)*: estimated cost from n to goal
- **Complete:** Yes (if h(n) is admissible)
- **Optimal:** Yes (with admissible and consistent h(n))
- **Time/Space Complexity:** Depends on heuristic quality

Use Case: Route planning (e.g., Google Maps), games, robotics

Summary:

Algorithm	Complete	Optimal	Data Structure	Use Case
BFS	□	□ (equal cost)	Queue	Shortest path
DFS	□	□	Stack	Memory-efficient exploration
A*	□	□	Priority Queue	Smart pathfinding